FlexQL Design Document

github.com/RounakNeogy/flexQL

Rounak Neogy (25CS60R55)

April 7, 2026

Contents

1 Introduction **4**
1.1 What is FlexQL? . . . 4
1.2 Problem Statement . . . 4
1.3 Project Scope . . . 4
1.4 Goals and Non-Goals . . . 5

2 Motivation **5**
2.1 Why this system was built . . . 5
2.2 Why performance is prioritized . . . 6
2.3 Why a client-server architecture . . . 6

3 System Overview **6**
3.1 High-level architecture . . . 6
3.2 Client responsibilities . . . 7
3.3 Server responsibilities . . . 7
3.4 Request-response flow . . . 8

4 Core Design Decisions **8**
4.1 Data storage model . . . 8
4.2 Row representation and serialization format . . . 9
4.3 Page layout and disk organization . . . 9
4.4 Schema and metadata management . . . 9
4.5 Persistence and durability model . . . 9
4.6 Fault tolerance choices . . . 10
4.7 Trade-offs considered . . . 10

5 Detailed Design **10**
5.1 How data is stored . . . 10
5.1.1 Physical storage format . . . 10
5.1.2 In-memory structures . . . 11
5.1.3 Catalog and schema persistence . . . 11
5.2 Indexing method . . . 12
5.2.1 Primary key indexing strategy . . . 12
5.2.2 B+ Tree structure and operations . . . 12
5.2.3 Bulk index build path . . . 12
5.3 Handling expiration timestamps . . . 13
5.3.1 Write-time semantics . . . 13
5.3.2 Read-time filtering . . . 13
5.3.3 Background expiration cleanup . . . 13

5.3.4 Compaction policy . . . 14
5.4 Multithreading design . . . 14
5.4.1 Thread pool . . . 14
5.4.2 Per-table locking . . . 14
5.4.3 Buffer pool synchronization . . . 15
5.4.4 WAL concurrency model . . . 15
5.4.5 Deadlock prevention and lock ordering . . . 15
5.5 Query processing pipeline . . . 15
5.5.1 Parser and AST . . . 15
5.5.2 Query planner . . . 16
5.5.3 Execution engine . . . 16
5.5.4 JOIN strategy selection . . . 16
5.6 Write-Ahead Logging . . . 17
5.6.1 WAL record format . . . 17
5.6.2 Group commit . . . 17
5.6.3 Checkpointing . . . 17
5.6.4 Recovery procedure . . . 17

6 External Interface **18**
6.1 C API design . . . 18
6.2 TCP binary protocol . . . 18
6.3 REPL behavior . . . 19

7 Technology Stack **19**
7.1 Programming language and standard . . . 19
7.2 Core libraries and system calls . . . 19
7.3 Build system and tooling . . . 20
7.4 Testing and benchmarking setup . . . 20

8 Performance Evaluation **20**
8.1 Benchmark methodology . . . 20
8.2 Test environment . . . 21
8.3 Insertion throughput results . . . 21
8.4 Selection latency results . . . 21
8.5 Memory footprint observations . . . 22

9 Challenges Faced **22**
9.1 Performance bottlenecks . . . 22
9.2 Correctness vs speed trade-offs . . . 22
9.3 Concurrency and synchronization issues . . . 23
9.4 Persistence and recovery edge cases . . . 23

10 Limitations **23**
10.1 Current SQL feature limitations . . . 23
10.2 Storage and scaling limitations . . . 24
10.3 Operational limitations . . . 24
10.4 Known performance constraints . . . 24

11 Future Work **25**
11.1 Query optimizer improvements . . . 25
11.2 Secondary indexing . . . 25
11.3 Advanced transaction support . . . 25

11.4 Compression and storage optimizations . . . 26
11.5 Distributed and scalable architecture . . . 26
11.6 Observability and monitoring . . . 26

12 Conclusion **26**
12.1 Summary of contributions . . . 26
12.2 Key takeaways . . . 27
12.3 Final remarks . . . 27

A Appendix A: API Summary **27**

B Appendix B: File and Module Layout **28**

C Appendix C: Reproducible Build and Run Commands **29**

D Appendix D: Additional Benchmark Logs **29**

1 Introduction

1.1 What is FlexQL?

FlexQL is a lightweight, high-performance, SQL-like database system implemented in C++ using a client–server architecture. It is designed to support a carefully selected subset of relational database operations while exposing both a C API for programmatic access and a REPL-based client for interactive querying.

The system implements core SQL functionalities, including `CREATE TABLE`, `INSERT`, `SELECT` (with projection and `WHERE` clauses), and `INNER JOIN`. Despite its compact scope, FlexQL is engineered with key database system principles found in production-grade engines. These include fixed-size paged storage, a buffer pool cache, B+ Tree indexing, write-ahead logging (WAL), a query planning layer, and multithreaded request handling.

The design philosophy emphasizes achieving high performance and persistence guarantees while maintaining conceptual simplicity, enabling the system to scale efficiently to datasets on the order of 10 million rows while remaining fully understandable end-to-end.

1.2 Problem Statement

The objective of this project is to design and implement a simplified SQL-like database engine from scratch in C/C++, without relying on existing database systems. The system must satisfy the following requirements:

- Parse and execute a defined subset of SQL queries.
- Ensure persistent storage on disk, with RAM used only as a cache.
- Support multiple concurrent clients via a multithreaded server architecture.
- Improve query performance through indexing and caching mechanisms.
- Handle row-level expiration semantics and visibility rules.
- Provide a C API (`flexql_open`, `flexql_exec`, `flexql_close`, `flexql_free`) compatible with the benchmark harness.

The primary challenge lies in balancing correctness, durability, and high performance under benchmark workloads involving large-scale inserts and queries over datasets of up to 10 million rows.

1.3 Project Scope

FlexQL focuses on implementing the essential components of a database engine required for the assignment and benchmarking workflow. The scope includes:

- A SQL-like parser and abstract syntax tree (AST) representation for supported queries.
- A row-oriented storage engine using fixed-size paged disk layout.
- A buffer pool with an eviction policy and efficient page access mechanisms.
- A primary-key B+ Tree index to accelerate point lookups.
- Sequential scan execution for non-indexed predicates.
- A query planner to choose between index scans and full table scans.

- A multithreaded TCP server with a binary-framed communication protocol.
- A client-facing C API and REPL interface.
- Write-ahead logging (WAL) with grouped commit behavior and crash recovery support.

The following features are intentionally out of scope:

- Full SQL standard compliance, including nested queries and complex boolean logic.
- Advanced transaction management with strict isolation guarantees.
- Cost-based query optimization and statistics-driven execution planning.
- Distributed features such as replication, sharding, or consensus protocols.
- Graphical user interfaces or administrative dashboards.

1.4 Goals and Non-Goals

Goals

- Build a persistent and fault-tolerant SQL-like engine for the defined feature subset.
- Achieve high insert throughput and low-latency query execution for benchmark workloads.
- Ensure compatibility with the provided testing and benchmarking infrastructure.
- Maintain a modular and extensible architecture to support future enhancements.
- Provide stable and predictable behavior under concurrent client workloads.

Non-Goals

- Full ANSI SQL compliance, including advanced query constructs and DDL/DML features.
- A complete OLTP transaction system with strict serializability guarantees.
- Advanced optimizer techniques such as join reordering or cost-based plan selection.
- Distributed database capabilities (replication, partitioning, or fault-tolerant clustering).
- Rich user interfaces; interaction is limited to API and terminal-based clients.

2 Motivation

2.1 Why this system was built

FlexQL was developed to gain a deep, first-principles understanding of how modern database systems are constructed, rather than relying on existing DBMS implementations. The primary objective is educational and systems-oriented: to design and implement the core components of a database engine, including the storage layer, query processing pipeline, indexing structures, caching mechanisms, and concurrency model, entirely in C++.

By building the complete execution pipeline — from parsing (→ AST construction → planning → execution → storage and networking) — the system provides full control over data layout, memory access patterns, synchronization primitives, and I/O behavior. This end-to-end ownership enables precise reasoning about performance characteristics and system correctness.

A key motivation is also transparency. Unlike production systems that often abstract away internal decisions, FlexQL makes each subsystem explicit and inspectable. Design choices such as page layout, row encoding, write-ahead logging (WAL), buffer management policies, and locking strategies are directly visible and measurable, allowing systematic evaluation of trade-offs.

2.2 Why performance is prioritized

Performance is a central design goal due to the benchmark-driven nature of the assignment, which evaluates both insertion throughput and query latency on datasets of up to 10 million rows. At this scale, inefficient design decisions quickly become critical bottlenecks. For example:

- Excessive heap allocations significantly reduce insert throughput.
- Synchronous per-row disk writes introduce severe I/O latency.
- Full table scans for point queries lead to unacceptable query times.
- Poor cache locality increases CPU overhead and memory access costs.

To address these challenges, FlexQL adopts several performance-oriented design strategies:

- Fixed-width binary row representation in performance-critical paths.
- Page-aligned on-disk storage to match operating system I/O granularity.
- A buffer pool cache to minimize repeated disk access.
- Primary-key indexing using a B+ Tree for efficient point lookups.
- Batched writes and grouped durability to amortize disk synchronization costs.

These choices aim to deliver high sustained throughput and low-latency query execution, while preserving correctness and durability guarantees under demanding workloads.

2.3 Why a client-server architecture

FlexQL adopts a client–server architecture to align with real-world database deployment models and to satisfy the assignment's interface requirements. The provided C API is designed for network-based interaction, necessitating a dedicated server process capable of handling multiple concurrent client connections.

This architectural separation introduces clear boundaries between system components:

- **Client side**: API wrapper, query submission, callback handling, REPL interface, and result presentation.
- **Server side**: query parsing and execution, storage management, indexing, caching, and concurrency control.

Such separation improves modularity, simplifies testing, and enables concurrent workloads from independent clients without embedding database logic into application processes. Additionally, it makes communication protocols, request framing, and callback semantics explicit, which is essential for benchmarking, debugging, and integration testing.

3 System Overview

3.1 High-level architecture

FlexQL follows a layered client–server architecture, with communication between components handled via a binary TCP protocol. The client provides a C API and an interactive REPL, while the server encapsulates the complete database engine.

Internally, the server is structured as a pipeline of well-defined components:

1. **Network layer**: Handles incoming TCP connections, decodes framed query messages, and streams framed responses (rows or errors) back to clients.
2. **Parser/AST layer**: Translates SQL-like input into structured internal representations (AST), ensuring syntactic correctness.
3. **Planner layer**: Determines an execution strategy (e.g., index scan vs. full table scan) based on query structure and available metadata.
4. **Execution layer**: Executes queries (inserts, selects, joins) using runtime operators, indexes, and table schemas.
5. **Storage layer**: Manages on-disk data organization, fixed-size pages, and buffer pool caching.
6. **Durability layer**: Ensures persistence through write-ahead logging (WAL), checkpointing, and recovery mechanisms.

This modular decomposition enables independent optimization of each subsystem while maintaining a clear, linear control flow from query ingestion to result generation.

3.2 Client responsibilities

The client is intentionally lightweight and acts as a thin API wrapper over the server. Its responsibilities include:

- Establishing and terminating TCP connections to the FlexQL server.
- Encoding and sending SQL queries as binary-framed protocol messages.
- Receiving schema metadata and row data as streamed frames.
- Converting binary-encoded values into user-friendly representations.
- Invoking user-defined callbacks once per result row.
- Propagating execution errors through the API's error-handling interface.

The interactive REPL is implemented entirely on top of this C API and does not bypass the protocol layer. This ensures a uniform interface across automated benchmarks, programmatic usage, and manual interaction.

3.3 Server responsibilities

The server is responsible for all core database functionality and maintains all shared system state. Its responsibilities include:

- Managing multiple concurrent client connections using a multithreaded execution model.
- Parsing and validating incoming SQL-like queries.
- Executing supported operations (`CREATE TABLE`, `INSERT`, `SELECT`, `JOIN`, `DELETE`).
- Maintaining schema definitions, metadata, and persistent storage files.
- Managing the buffer pool and implementing page eviction policies.
- Maintaining indexing structures (e.g., primary-key B+ Tree).

- Enforcing row visibility rules, including expiration and tombstone handling.
- Ensuring durability via WAL and enabling recovery after crashes.

Additionally, the server is responsible for encoding responses into protocol frames and ensuring correctness and isolation under concurrent workloads.

3.4 Request-response flow

A typical request-response lifecycle in FlexQL proceeds as follows:

1. The client sends a `QUERY` frame containing SQL text.
2. The server decodes the frame, parses the query into an AST, and validates schema references.
3. The planner selects an execution strategy based on query type and metadata.
4. The execution engine evaluates the plan:
   - Write operations pass through insertion logic, indexing, and durability mechanisms.
   - Read operations perform either index-based lookups or buffer-assisted table scans.
5. For `SELECT` queries, result rows are streamed incrementally back to the client.
6. The client decodes each row, converts values, and invokes the registered callback.
7. The server sends a terminal `DONE` frame upon completion (or an `ERROR` frame if execution fails).

This streaming execution model avoids full result materialization, enabling lower latency and improved responsiveness for large result sets.

4 Core Design Decisions

4.1 Data storage model

FlexQL adopts a **row-oriented** storage model with **paged disk storage** as the primary source of truth. This design is motivated by the characteristics of the target workload:

- Insert-heavy and point-lookup workloads naturally align with row-major storage.
- A row-wise layout simplifies serialization, WAL replay, and row delivery to client callbacks.
- Fixed-size page I/O integrates efficiently with operating system and filesystem behavior.

Each table is persisted under `data/<TABLE_NAME>/`, where disk files represent the authoritative state. In-memory structures such as the buffer pool, indexes, and insert buffers act purely as performance accelerators and can be reconstructed from persistent storage.

4.2 Row representation and serialization format

To minimize overhead in performance-critical paths, FlexQL represents rows in a compact binary format (`RowValue`), avoiding repeated string conversions during execution. Each row consists of:

- A fixed-size header (tombstone flag and expiration timestamp).
- Column values serialized sequentially in schema order.

Type-specific encodings are chosen for efficiency:

- `INT`, `DATETIME`: 8-byte signed integers.
- `DECIMAL`: 8-byte IEEE-754 double precision.
- `VARCHAR`: length-prefixed byte arrays (avoiding server-side formatting in hot paths).

This layout enables direct pointer arithmetic for column access during scans, significantly reducing interpretation overhead and improving cache locality.

4.3 Page layout and disk organization

FlexQL uses fixed-size pages (4KB) consisting of a compact header and a contiguous region for row storage. The layout is designed to achieve:

- Deterministic offset computation for fast access.
- Efficient append operations for high-throughput inserts.
- Simplified buffer pool integration with page-level caching and eviction.

On disk, each table is divided into segment files (e.g., `segment_0.db`), each containing a sequence of serialized pages. This structure ensures predictable storage growth and aligns with buffer pool semantics (fetch, pin, unpin).

4.4 Schema and metadata management

Schema information is maintained in both memory and persistent storage:

- **In-memory**: Column definitions, type metadata, and name-to-index mappings for fast query resolution.
- **On-disk**: Catalog metadata and per-table schema snapshots for recovery.

During parsing, column references are resolved into indices and precomputed offsets. This eliminates repeated name-based lookups during execution, enabling efficient inner-loop processing.

4.5 Persistence and durability model

FlexQL ensures durability through a combination of:

- **Write-Ahead Logging (WAL)** to record changes before applying them to data pages.
- **Group commit** to batch multiple writes into fewer disk synchronization operations.
- **Checkpointing** to periodically persist in-memory state and limit WAL replay length.

This approach allows newly inserted data to become visible quickly while amortizing disk I/O costs, achieving a balance between high throughput and reliable crash recovery.

4.6 Fault tolerance choices

Fault tolerance is designed to provide practical recovery guarantees without excessive complexity:

- WAL replay on startup restores committed changes since the last checkpoint.
- Persistent catalog metadata enables reconstruction of table schemas.
- Strict protocol framing and input validation prevent corruption from malformed client requests.
- Explicit flush mechanisms ensure consistency during controlled shutdown.

The system prioritizes recoverability and consistency for the target workload, rather than full transactional isolation or advanced failure handling.

4.7 Trade-offs considered

Several deliberate trade-offs guide the system design:

- **Binary performance vs. human readability**: Internal storage uses compact binary formats for speed, while debug-friendly representations are handled separately.
- **Feature scope vs. implementation depth**: A limited SQL subset enables deeper optimization of supported operations.
- **Throughput vs. strict durability**: Grouped WAL commits improve performance but introduce a small window before data is fully persisted.
- **Index simplicity vs. query flexibility**: A primary-key B+ Tree is prioritized for fast point lookups, while non-indexed queries fall back to scans.
- **Performance vs. system complexity**: Components such as the planner, buffer pool, WAL, and concurrency control are modularized to balance efficiency with maintainability.

5 Detailed Design

5.1 How data is stored

5.1.1 Physical storage format

FlexQL persists table data using a paged, row-oriented storage layout. Each table is stored under `data/<TABLE_NAME>/`, with the primary data maintained in binary segment files (e.g., `segment_0.db`). The storage is organized into fixed-size pages (4KB), each comprising:

- A compact page header containing metadata such as row count and free-space offset.
- A contiguous data region storing serialized rows.

Rows are encoded in a compact binary format rather than textual SQL representation. Each row consists of:

- A tombstone flag to indicate logical deletion or expiration.
- An expiration timestamp for visibility control.

- Column values serialized sequentially in schema order using fixed binary encoding.

This design provides several advantages:

- Efficient append-only behavior for high-throughput inserts.
- Direct offset-based access enabling fast scans and projection.
- Compatibility with low-level I/O primitives such as `pread`/`pwrite` for predictable disk access.

5.1.2 In-memory structures

While disk remains the authoritative data store, FlexQL leverages multiple in-memory structures to accelerate execution:

1) Table runtime state

- Insert buffers to batch incoming writes and reduce I/O overhead.
- Primary-key helper maps for fast lookup of recently inserted (unflushed) rows.
- A B+ Tree index for efficient lookup of persisted rows.
- Page-level Bloom filters built dynamically to prune unnecessary disk access during scan-based queries.

2) Buffer pool

- Caches frequently accessed pages in memory to reduce disk I/O.
- Tracks page metadata including dirty status, pin count, and access history.
- Employs an eviction policy that prioritizes unpinned clean pages, and flushes dirty pages when necessary.

3) Query-time transient structures

- Parsed AST and execution plans with precomputed column indices and offsets.
- Temporary buffers/vectors for storing projected row values during execution.

The system is designed such that critical execution paths operate on compact binary representations, minimizing object allocation and reducing CPU overhead.

5.1.3 Catalog and schema persistence

FlexQL maintains persistent metadata to ensure that database state can be reconstructed after restart. This includes:

Global catalog

- Stored in `data/catalog.txt`.
- Contains table identifiers, names, and serialized column definitions.

Per-table schema snapshot

- Stored in `data/<TABLE_NAME>/schema.txt`.
- Provides a human-readable schema representation for inspection and recovery.

During server startup, the catalog is loaded into memory, tables are reconstructed, and storage files are reopened. Runtime structures such as indexes and caches are rebuilt or warmed using persisted data and recovery logs as needed, ensuring consistency with the last durable state.

5.2 Indexing method

5.2.1 Primary key indexing strategy

FlexQL employs a **single primary-key index** per table to accelerate point queries of the form `WHERE pk = value`. To minimize runtime overhead, primary keys are encoded into a compact numeric representation at insertion time, ensuring that lookup operations avoid expensive conversions:

- `INT`, `DATETIME`: stored directly as 64-bit integers.
- `DECIMAL`: encoded as 64-bit IEEE-754 values via bit-casting.
- `VARCHAR`: mapped to a 64-bit hash; collisions are resolved by verifying the original value at the leaf level.

This design ensures fast key comparisons and avoids string-based operations in critical lookup paths. Queries on non-primary attributes are evaluated using scan-based execution.

5.2.2 B+ Tree structure and operations

The primary-key index is implemented as an in-memory B+ Tree, optimized for high fan-out and shallow depth to minimize lookup latency:

- **Internal nodes**: store separator keys and child pointers.
- **Leaf nodes**: store mappings from keys to record pointers.
- **Record pointers**: reference the physical location of rows as `(page_id, row_offset)` in paged storage.

The B+ Tree supports the following core operations:

- **Insert**: Inserts a key–pointer pair upon row persistence. Node splits are performed when capacity is exceeded, with split keys propagated upward to maintain tree balance.
- **Search**: Traverses from root to leaf to locate the target key, returning the corresponding record pointer. For hashed `VARCHAR` keys, collisions are resolved via value verification.
- **Rebuild/Clear**: Reconstructs or resets the index during recovery, compaction, or table reinitialization.

To reduce allocation overhead and improve cache locality, the implementation uses a node-pool allocation strategy for managing tree nodes.

5.2.3 Bulk index build path

To handle high-throughput ingestion workloads, FlexQL provides a bulk-oriented index construction path. This approach minimizes per-row index maintenance overhead during large insert batches while ensuring that the index remains consistent and queryable after ingestion.

The bulk loading process follows these steps:

1. Rows are persisted sequentially into pages, with key and record pointer metadata captured during insertion.
2. Index updates are applied in a batched or amortized manner to reduce structural modifications and rebalancing overhead.

3. Upon completion, the index is finalized and transitions back to standard incremental update mode.

This strategy significantly improves ingestion throughput while preserving efficient primary-key lookup performance immediately after bulk operations.

5.3 Handling expiration timestamps

5.3.1 Write-time semantics

Each row inserted into FlexQL carries an `expiration_timestamp`, stored as a 64-bit epoch value in the row header. The semantics are defined as follows:

- `0`: The row does not expire and remains valid until explicitly deleted or tombstoned.
- `> 0`: The row is valid only until the specified timestamp.

The expiration value is persisted alongside row data within page storage and is also recorded in the write-ahead log (WAL). This ensures that row visibility semantics remain consistent across crashes and recovery.

5.3.2 Read-time filtering

Expiration is enforced during query execution, prior to predicate evaluation and result projection. For each candidate row, the execution engine applies the following checks:

1. Skip the row if it is marked as tombstoned.
2. Read the expiration timestamp from the row header.
3. Skip the row if `expiry > 0` and `expiry < now()`.
4. Evaluate `WHERE` or `JOIN` predicates only on rows that are both non-tombstoned and non-expired.

This design guarantees logical correctness independently of physical cleanup. Expired rows are never visible to queries, even if they have not yet been physically removed from storage.

5.3.3 Background expiration cleanup

FlexQL employs a background maintenance thread to reclaim storage occupied by expired rows. This process operates asynchronously and performs the following steps:

- Iterates over table pages via the buffer pool.
- Identifies rows whose expiration timestamps have elapsed.
- Marks such rows as tombstoned in-place and flags the corresponding pages as dirty.

Since read-time filtering already ensures correctness, this cleanup mechanism is an optimization for storage efficiency and scan performance rather than a requirement for correctness.

5.3.4 Compaction policy

To mitigate the accumulation of dead space, FlexQL triggers compaction when the fraction of tombstoned rows exceeds a configurable threshold (e.g., ∼20%). The compaction process proceeds as follows:

1. Flush all pending insert buffers and bulk operations to disk immediately.
2. Scan existing segment files and rewrite only live (non-expired, non-tombstoned) rows into a new segment.
3. Skip all invalid rows during the rewrite phase.
4. Rebuild the primary-key B+ Tree index and all page-level Bloom filters using the compacted dataset.
5. Atomically replace the old segment files with the compacted version.

Compaction improves storage utilization and reduces scan overhead by eliminating dead rows. Rebuilding the indexes and Bloom filters ensures that all state remains consistent with the new physical layout.

5.4 Multithreading design

5.4.1 Thread pool

FlexQL employs a bounded worker thread pool to handle concurrent server tasks, including client request processing and query execution. The pool size is capped to prevent excessive context switching and resource contention under high load.

Core characteristics of the design include:

- Tasks are submitted to a synchronized work queue.
- Worker threads block on a condition variable when no work is available.
- The shutdown procedure ensures graceful draining of pending tasks and clean thread termination.

This model provides controlled parallelism and avoids the overhead and instability associated with spawning one OS thread per query.

5.4.2 Per-table locking

To balance concurrency and correctness, FlexQL adopts **table-level reader–writer locks**:

- **Read operations** (`SELECT`) acquire shared locks, allowing multiple concurrent readers.
- **Write operations** (`INSERT`, `DELETE`, and structural changes) acquire exclusive locks.

Using per-table locks, rather than a global database lock, enables concurrent operations across independent tables while ensuring consistency of each table's storage, index, and runtime state.

5.4.3 Buffer pool synchronization

The buffer pool employs a hierarchical synchronization strategy to minimize contention:

- A global mutex protects access to the page table and frame registry.
- Each buffer frame maintains a dedicated mutex for protecting mutable state (e.g., dirty flag, pin count, access metadata).

This separation allows high concurrency for cache-hit paths while keeping critical sections small. Disk I/O operations are performed outside global locks wherever possible to avoid blocking unrelated threads.

5.4.4 WAL concurrency model

The write-ahead logging (WAL) subsystem follows a double-buffered, group-commit design:

- Foreground threads append log records to an in-memory WAL buffer under short-duration locks.
- A background flush thread periodically swaps the active buffer with a flush buffer and performs disk writes and `fsync` operations asynchronously.

This design eliminates per-operation disk synchronization overhead while preserving ordering guarantees at batch boundaries, achieving high write throughput with durable persistence.

5.4.5 Deadlock prevention and lock ordering

To prevent deadlocks, FlexQL enforces a strict global lock acquisition hierarchy. Locks are always acquired in a predefined order, moving from coarse-grained to fine-grained components, and never in reverse:

- Database and catalog metadata locks,
- Table-level reader–writer locks,
- Buffer pool global locks,
- Per-frame locks,
- WAL internal locks.

This consistent ordering eliminates cyclic wait conditions across subsystems. During development, optional debug assertions can be used to validate lock acquisition order and detect violations early. This policy is critical for maintaining correctness under mixed concurrent workloads involving reads, writes, and background maintenance operations.

5.5 Query processing pipeline

5.5.1 Parser and AST

Incoming SQL-like queries are first normalized and parsed into a typed Abstract Syntax Tree (AST). The parser supports the core subset of statements:

- `CREATE TABLE`
- `INSERT`

- `SELECT` (projection with optional `WHERE`)
- `INNER JOIN`
- `DELETE FROM`

During parsing, column references are resolved against schema metadata, and literal values are converted into typed internal representations. To improve execution efficiency, the parser annotates the AST with resolved column indices and precomputed metadata (e.g., projection lists and predicate operands), eliminating repeated name-based lookups during execution.

5.5.2 Query planner

The query planner transforms the AST into an executable plan by selecting an appropriate access strategy based on query structure and available indexing:

- **Index scan**: selected for primary-key equality predicates.
- **Full table scan**: used for non-indexed predicates or queries without filters.

The planner also propagates projection metadata, including column indices and offsets, allowing the execution engine to read only the required fields. This reduces per-row processing overhead and improves cache efficiency during scans.

5.5.3 Execution engine

The execution engine evaluates query plans by interacting with storage, indexing, and runtime components:

- **Insert path**: Validates input, appends rows via the buffered write path, updates index structures, and coordinates with WAL and checkpointing for durability.
- **Select path**: Retrieves rows from insert buffers, indexes, or paged storage depending on the access strategy; enforces tombstone and expiration visibility rules; and streams projected results incrementally.
- **Delete/reset path**: Ensures consistency by flushing pending state, updating metadata, and resetting relevant storage and index structures.

Read paths are optimized for sequential page access and offset-based field extraction, while write paths prioritize batching and amortized durability costs.

5.5.4 JOIN strategy selection

For `INNER JOIN` operations, FlexQL selects the execution strategy based on index availability and join characteristics:

- **Indexed nested-loop join**: Used when the inner relation can be efficiently probed via the primary-key index.
- **Hash join**: Used when no suitable index is available on the join key.

In both cases, predicate filtering is applied as early as possible to reduce intermediate result size. Join-key comparisons are performed on typed internal representations rather than string formats, minimizing CPU overhead and improving execution efficiency.

5.6 Write-Ahead Logging

5.6.1 WAL record format

FlexQL adopts a compact binary WAL format to minimize CPU overhead and parsing cost under high write throughput. Each WAL record is encoded as:

- Record type (e.g., INSERT, CHECKPOINT),
- Payload length,
- Typed payload bytes.

INSERT records contain all information required to reconstruct a row, including sequence number, table identifier, expiration metadata, and schema-consistent typed column values. **CHECKPOINT** records store the durable sequence boundary.

Compared to text-based logging, the binary format reduces serialization overhead, improves cache efficiency, and enables faster replay by avoiding string parsing.

5.6.2 Group commit

To mitigate the cost of synchronous disk writes, FlexQL employs a group commit strategy:

- Foreground threads append WAL records to an in-memory active buffer under short-duration locks.
- A background flush thread periodically (or based on size thresholds) swaps the active buffer with a flush buffer.
- The flush buffer is written to disk, followed by a single `fsync` to commit the batch.

This design amortizes disk synchronization costs across multiple operations, significantly improving insert throughput while maintaining durability guarantees at batch boundaries.

5.6.3 Checkpointing

Checkpointing is used to bound WAL growth and reduce recovery time. During a checkpoint:

1. Dirty pages from the buffer pool are flushed to persistent storage.
2. A **CHECKPOINT** record is appended to the WAL, marking a consistent durability boundary.
3. WAL segments prior to this boundary may be truncated or compacted to limit replay overhead.

Checkpointing is triggered based on write volume thresholds and lifecycle events (e.g., clean shutdown), balancing runtime overhead with recovery efficiency.

5.6.4 Recovery procedure

On startup, FlexQL performs recovery to restore a consistent database state:

1. Load catalog and schema metadata, and reopen table storage files.
2. Identify the most recent checkpoint boundary in the WAL.

3. Replay WAL records beyond the checkpoint, reapplying insert operations and rebuilding in-memory structures (e.g., indexes).

Replay is designed to be idempotent relative to checkpoint boundaries, ensuring that partially applied operations do not corrupt state. This process restores all committed data that may not have been fully persisted to table storage prior to an unclean shutdown. Once recovery completes, the system resumes normal operation with consistent storage and index structures.

6 External Interface

6.1 C API design

FlexQL exposes a C-compatible client API that allows external applications to interact with the database server without linking directly to internal engine components. The API is centered around an opaque database handle and a minimal set of core functions:

- `flexql_open(host, port, db)`: Establishes a connection to the server and initializes a database handle.
- `flexql_exec(db, sql, callback, arg, errmsg)`: Executes a SQL query and streams results via a row-level callback.
- `flexql_close(db)`: Closes the connection and releases associated resources.
- `flexql_free(ptr)`: Frees memory allocated by the API (e.g., error messages).

The API is designed with the following principles:

- **Stable interface**: Provides an ABI-like contract compatible with the benchmark harness.
- **Explicit error handling**: Uses clear return codes (`FLEXQL_OK`, `FLEXQL_ERROR`) and error-message propagation.
- **Streaming execution**: Delivers results row-by-row via callbacks, enabling efficient handling of large result sets.
- **Encapsulation**: Hides all server-side implementation details from the client.

6.2 TCP binary protocol

Communication between client and server is implemented using a framed binary protocol to ensure efficiency, correctness, and unambiguous message boundaries. Each message follows the format:

```
[4-byte payload length][1-byte message type][payload bytes]
```

The protocol supports multiple message categories, including query submission, incremental row streaming, completion signaling, error reporting, schema metadata exchange, and session control.

Using a binary framing scheme eliminates the overhead and ambiguity of text-based protocols, particularly for large result sets. Row values are transmitted in compact typed binary form, with conversion to human-readable strings deferred to the client when required for display or callback processing.

6.3 REPL behavior

FlexQL includes a REPL (Read-Eval-Print Loop) client built entirely on top of the public C API, without direct access to server internals. Its behavior is as follows:

- Establish a connection to the server using host and port parameters.
- Continuously read SQL input from the terminal.
- Execute queries via `flexql_exec`.
- Display results in a tabular format using returned schema information.
- Support graceful termination via commands such as `\quit`.

By relying exclusively on the C API, the REPL serves both as a user-facing interface and as a validation tool for end-to-end system behavior. It ensures consistency across interactive usage, automated benchmarks, and external integrations.

7 Technology Stack

7.1 Programming language and standard

FlexQL is implemented in **C++17**, chosen for its balance between low-level control and modern abstractions:

- Fine-grained control over memory layout and data representation, critical for storage, indexing, and caching paths.
- Rich standard library support for containers, concurrency, and utilities.
- Seamless interoperability with a C-compatible API surface.

The external interface is exposed via a C-compatible header (`flexql.h`), enabling integration with benchmark harnesses and external applications without requiring direct linkage to C++ internals.

7.2 Core libraries and system calls

FlexQL relies primarily on the C++ standard library alongside POSIX APIs for networking and file system interaction.

C++ standard library components:

- **Containers**: `std::vector`, `std::unordered_map`, `std::deque`, `std::string`
- **Concurrency**: `std::thread`, `std::mutex`, `std::shared_mutex`, `std::condition_variable`, atomic types
- **Utilities**: `std::chrono`, `std::optional`, standard algorithms, `std::to_chars`

POSIX/system APIs:

- **File I/O and durability**: `open`, `close`, `pread`, `pwrite`, `fsync`
- **Networking**: `socket`, `bind`, `listen`, `accept`, `connect`, `send`, `recv`, `shutdown`
- **Socket options**: `SO_REUSEADDR`, `TCP_NODELAY`

This combination ensures portability across Unix-like systems (Linux/macOS) while maintaining precise control over performance-critical operations.

7.3 Build system and tooling

FlexQL uses a **Makefile-based** build system for simplicity, transparency, and reproducibility:

- Dedicated targets for server, REPL client, and benchmarking binaries.
- Configurable runtime parameters (e.g., host, port, dataset size, client count) via Make variables.
- Utility targets for cleaning build artifacts and resetting data directories.

Typical compilation settings include:

- Compilers: `g++` or `clang++`
- Flags: `-std=c++17 -O2 -pthread`

The REPL optionally links against `readline` or `editline`, depending on platform availability, for improved interactive usability.

7.4 Testing and benchmarking setup

FlexQL is validated through a combination of correctness testing and performance benchmarking:

Correctness validation:

- Unit-level verification of supported SQL operations (table creation, insert, filtering, joins, and error handling).
- API-level validation of callback behavior and error propagation.
- Persistence testing across restarts to ensure durability guarantees.

Performance evaluation:

- Benchmark harness implemented in `benchmark_flexql.cpp`.
- Measurement of multi-client insert throughput under concurrent workloads.
- Evaluation of query latency for full scans and indexed lookups.
- Large-scale testing with datasets up to 10 million rows.

All testing and benchmarking workflows utilize the same C API, ensuring consistency between evaluation scenarios and real-world usage.

8 Performance Evaluation

8.1 Benchmark methodology

Performance evaluation was conducted using the provided benchmark driver (`benchmark_flexql.cpp`) interacting with the FlexQL server via the C API over TCP. The benchmark includes:

- Multi-client batched `INSERT` workload into the `BIG_USERS` table.
- Multi-client full-table `SELECT` workload.
- Multi-client primary-key `WHERE` lookup workload.

- SQL subset correctness validation (unit-test suite).

Unless otherwise specified, experiments were performed under the following configuration:

- Dataset size: up to 10,000,000 rows.
- Concurrency: 4 parallel client connections.
- Insert mode: batched `INSERT` statements with large `VALUES` lists.

The benchmark reports elapsed execution time (in milliseconds) and derived throughput (rows per second).

8.2 Test environment

All experiments were executed on a local development setup using a client–server model:

- A single FlexQL server process running on localhost.
- A benchmark driver spawning multiple concurrent client connections.
- Persistent, disk-backed storage enabled using segment files.

The system was compiled using optimized settings (e.g., `-O2`, C++17, `-pthread`), ensuring results reflect near-production performance rather than debug behavior.

8.3 Insertion throughput results

For a 10M-row workload, FlexQL achieved:

- **Elapsed time:** `28,318 ms`
- **Throughput:** `353,132 rows/sec`

These results demonstrate the effectiveness of several design optimizations:

- Batched insert execution, reducing per-row overhead.
- Minimal synchronization in the critical insert path.
- Group commit in WAL, eliminating per-row `fsync` costs.
- Sequential page append behavior, improving I/O locality.

Overall, the system sustains high ingestion throughput while maintaining durability guarantees.

8.4 Selection latency results

For the same dataset, query performance was observed as:

- **Full table scan** (4 clients, 40M total rows returned): `18,660 ms`
- **Primary-key lookup** (4 clients, 4 total rows): `5 ms`

These results highlight two distinct access patterns:

- Full scans scale linearly with data volume and are dominated by row materialization and memory traversal costs.
- Primary-key queries achieve near-constant latency due to B+ Tree index lookup, demonstrating the effectiveness of the indexing strategy.

8.5 Memory footprint observations

Memory usage is primarily influenced by the following components:

- Buffer pool size (dominant contributor to memory usage).
- In-memory B+ Tree index structure.
- Insert buffers and transient query execution structures.

Observed behavior under benchmark conditions:

- Memory usage increases during the initial insert phase and stabilizes once steady-state is reached.
- Buffer pool reuse significantly reduces disk I/O during repeated scans.
- Avoidance of string-heavy representations in hot paths reduces allocator pressure and improves memory efficiency.

A detailed memory usage profile (e.g., max RSS over time) can be incorporated in future iterations using profiling tools.

9 Challenges Faced

9.1 Performance bottlenecks

As the system scaled from small test cases to 10M-row workloads, several performance bottlenecks became evident:

- **Per-row overhead in insert path**: Repeated validation, logging, and synchronization introduced significant CPU overhead at scale.
- **String-heavy execution paths**: Frequent string conversions in result processing increased latency in hot query paths.
- **Buffer pool contention**: Lock contention and unnecessary lock upgrades on cache-hit paths reduced throughput under concurrent workloads.
- **Scan inefficiency**: Initial scan implementations incurred avoidable per-row overhead due to redundant projection work and suboptimal memory access patterns.

These issues were mitigated through batching strategies, binary-oriented execution paths, reduced lock scope, and optimized scan loops with improved locality and minimal per-row work.

9.2 Correctness vs speed trade-offs

A key challenge was balancing performance optimizations with correctness and durability guarantees:

- **Durability vs throughput**: Strict per-operation `fsync` ensures strong durability but severely limits throughput; group commit improves performance at the cost of a bounded durability window.
- **Insert efficiency vs constraint enforcement**: Maintaining correct primary-key uniqueness checks without introducing expensive per-row validation logic required careful design.

- **Streaming vs completeness**: Streaming large result sets avoids excessive memory usage but introduces complexity in callback handling, partial result delivery, and early termination scenarios.

The final design prioritizes correctness-preserving optimizations such as batching, precomputation, and efficient typed comparisons, rather than sacrificing correctness for raw speed.

9.3 Concurrency and synchronization issues

Handling concurrent workloads introduced several challenges related to synchronization and system safety:

- Coordinating table-level locks with buffer pool and WAL synchronization without introducing deadlocks.
- Maintaining consistency between storage, index structures, and runtime state during overlapping read/write operations.
- Avoiding long critical sections, particularly around disk I/O operations.
- Ensuring correct behavior when client-side callbacks terminate result consumption prematurely.

These issues were addressed by enforcing a strict lock ordering policy, minimizing lock hold durations, and clearly separating synchronization domains across subsystems.

9.4 Persistence and recovery edge cases

Ensuring correctness under crash and restart scenarios revealed several subtle edge cases:

- Handling cases where persistent metadata was available but in-memory structures (e.g., indexes) required reconstruction.
- Correctly managing WAL replay boundaries to avoid duplicate application or data loss.
- Coordinating table reset and deletion workflows with storage, metadata, and runtime state updates.
- Managing checkpoint timing under mixed workloads to prevent large-table activity from negatively impacting smaller operations.

Robustness in these scenarios was achieved through repeated validation using benchmark workloads and unit tests, along with careful refinement of recovery and checkpoint logic.

10 Limitations

10.1 Current SQL feature limitations

FlexQL intentionally supports a constrained SQL-like subset, prioritizing depth of implementation over breadth of features. Current limitations include:

- Limited support for complex boolean expressions beyond the implemented predicate forms.
- A restricted DDL/DML surface compared to full-featured relational database systems.
- No support for nested queries, aggregation, grouping, window functions, or advanced SQL constructs.

- JOIN functionality limited to supported `INNER JOIN` patterns.
- A rule-based query planner without statistics-driven cost optimization.

These constraints allow focused optimization of core execution paths while keeping the system manageable and analyzable.

10.2 Storage and scaling limitations

The storage engine is designed for single-node, assignment-scale workloads and does not yet incorporate advanced scalability features:

- Simplified segment management, typically relying on a single dominant segment per table.
- No support for compression, tiered storage, or advanced space reclamation strategies.
- Lack of distributed capabilities such as partitioning, replication, or fault-tolerant clustering.
- Indexing is limited to primary-key access, with no generalized secondary indexing framework.

While sufficient for the target workload, these limitations would impact performance and manageability in larger or more diverse production environments.

10.3 Operational limitations

FlexQL provides minimal operational tooling, reflecting its focus as a systems implementation rather than a production-ready service:

- Limited observability, with no comprehensive metrics, tracing, or monitoring infrastructure.
- Manual lifecycle management, including startup, reset, and benchmarking workflows.
- No support for authentication, authorization, or encrypted (TLS) communication.
- Failure handling and recovery mechanisms are functional but not hardened for enterprise-grade reliability scenarios.

These constraints highlight areas where production systems typically invest heavily in tooling and reliability engineering.

10.4 Known performance constraints

Despite strong benchmark performance, certain architectural trade-offs introduce inherent limitations:

- Full-table scans remain expensive for non-indexed predicates due to the row-oriented design.
- Large result sets incur overhead in row materialization and client-side conversion.
- Durability mechanisms (WAL flushing and checkpointing) can introduce periodic latency spikes under sustained write pressure.
- Lock contention may arise under mixed read/write workloads, particularly around shared structures.

These limitations are consistent with the chosen design trade-offs and provide clear directions for future optimization and system evolution.

11 Future Work

11.1 Query optimizer improvements

The current rule-based planner can be extended into a more sophisticated cost-based optimizer:

- Maintain lightweight statistics (e.g., table cardinality, value distributions, selectivity estimates).
- Incorporate cost models to guide decisions between index scans, full scans, and join strategies.
- Enhance predicate pushdown and projection pruning across all operators.
- Introduce adaptive execution strategies that adjust plans based on runtime feedback (e.g., observed row counts, cache behavior).

Such improvements would enable more efficient query execution across diverse workloads and data distributions.

11.2 Secondary indexing

While FlexQL currently supports primary-key indexing, extending indexing capabilities is a natural next step:

- Enable creation of indexes on non-primary columns.
- Support composite indexes for multi-column query acceleration.
- Extend the planner to select optimal indexes among multiple candidates.
- Ensure consistent index maintenance and recovery integration across all write paths.

These enhancements would significantly improve performance for a wider range of query patterns.

11.3 Advanced transaction support

The existing durability mechanisms provide a foundation for richer transactional semantics:

- Support multi-statement transactions with `BEGIN`, `COMMIT`, and `ROLLBACK`.
- Introduce isolation levels (e.g., read committed, repeatable read, snapshot isolation).
- Incorporate MVCC or enhanced lock management for improved read-write concurrency.
- Provide configurable commit acknowledgment modes to balance latency and durability.

These features would move FlexQL closer to a full transactional database system.

11.4 Compression and storage optimizations

Storage efficiency and I/O performance can be further improved through:

- Page-level or column-aware compression for cold or archival data.
- Advanced space management with free-space tracking and reuse.
- Multi-segment compaction and merge policies for long-running workloads.
- Tunable asynchronous prefetching and writeback strategies for different storage devices.

Such optimizations would reduce storage footprint and improve long-term performance stability.

11.5 Distributed and scalable architecture

To support larger-scale deployments, FlexQL can evolve toward a distributed architecture:

- Implement hash- or range-based sharding across nodes.
- Introduce a coordinator layer for distributed query planning and routing.
- Add replication for fault tolerance and read scalability.
- Support rebalancing and failover mechanisms for cluster resilience.

These extensions would enable horizontal scalability and higher availability.

11.6 Observability and monitoring

Improved observability is essential for debugging, tuning, and production readiness:

- Expose internal metrics (e.g., QPS, latency percentiles, cache hit rate, WAL lag, checkpoint duration).
- Add structured logging with request-level tracing.
- Provide administrative endpoints for inspecting runtime state (indexes, buffer pool, query activity).
- Integrate profiling and telemetry pipelines for performance regression tracking.

Enhanced observability would significantly improve system operability and maintainability.

12 Conclusion

12.1 Summary of contributions

This project presents the design and implementation of FlexQL, a lightweight SQL-like database system built from first principles in C++. The system integrates core components of a modern database engine, including:

- A client–server architecture with a C-compatible API and efficient binary TCP protocol.
- A complete query processing pipeline comprising parsing, planning, and execution stages.
- Persistent paged storage with schema and catalog management.

- A buffer pool for caching and efficient page-level data access.
- Primary-key B+ Tree indexing with scan-based fallback for non-indexed queries.
- Write-ahead logging (WAL), checkpointing, and recovery for durability.
- Multithreaded execution with explicit synchronization and lock-order guarantees.

Collectively, these components demonstrate a functional and performant database driver capable of handling large-scale workloads while maintaining correctness and persistence guarantees.

12.2 Key takeaways

The development of FlexQL provided several important insights into database system design:

- Efficient data layout and memory access patterns are critical to performance, often outweighing purely algorithmic considerations.
- Batching and asynchronous durability mechanisms are essential for achieving high write throughput.
- Query performance depends heavily on early and appropriate access-path selection (index vs. scan).
- Concurrency correctness requires disciplined lock design and clear separation of synchronization domains.
- Persistence guarantees must be validated under failure and recovery scenarios, not just steady-state execution.

These lessons reflect fundamental principles applicable to real-world database and storage system development.

12.3 Final remarks

FlexQL is intentionally scoped to balance implementation complexity with performance and correctness requirements. Despite its limited feature set, it establishes a solid architectural foundation for a high-performance SQL-like engine.

The system demonstrates how core database concepts—storage layout, indexing, concurrency control, and durability—interact in practice. With further enhancements in query optimization, indexing capabilities, and operational tooling, FlexQL can evolve beyond an academic prototype into a more complete and production-oriented data system.

A Appendix A: API Summary

Public C API

- `int flexql_open(const char* host, int port, FlexQL_DB** db);`
- `int flexql_close(FlexQL_DB* db);`
- `int flexql_exec(FlexQL_DB* db, const char* sql,`
- `int (*callback)(void*, int, char**, char**),`
- `void* arg, char** errmsg);`
- `void flexql_free(void* ptr);`

Status Codes

- `FLEXQL_OK`: Operation completed successfully.
- `FLEXQL_ERROR`: Operation failed (error details returned via `errmsg`).

Callback Contract

- Invoked once per result row for `SELECT`-like queries.
- Signature:

```
int callback(void* data, int columnCount,
             char** values, char** columnNames);
```

- Return `0` to continue iteration, `1` to abort early.

B Appendix B: File and Module Layout

Top-level structure (representative)

- `include/flexql/` — public and internal headers
- `src/` — core engine, parser, planner, server, and API implementation
- `tests/` — correctness and latency test cases
- `FlexQL_Benchmark_Unit_Tests-main/benchmark_flexql.cpp` — benchmark harness
- `data/` — persistent catalog, table files, WAL logs
- `Makefile` — build and execution targets

Core modules

- `core_types.*` — schema definitions and row/value primitives
- `storage_engine.*` — page management and segment I/O
- `buffer_pool.*` — in-memory page caching and eviction
- `row_codec.*` — row serialization and deserialization
- `bplus_tree.*` — primary-key index implementation
- `query_parser.*` — SQL-like parsing and AST construction
- `query_planner.*` — access-path planning
- `execution_engine.*` — execution of insert/select/join operations
- `wal_manager.*` — WAL logging, flushing, checkpointing, and recovery
- `tcp_server.*` — network protocol handling and connection management
- `flexql_c_api.*` — client-side API implementation over TCP

C Appendix C: Reproducible Build and Run Commands

Build

```
make clean
make -j4 server repl benchmark
```

Run server

```
make run-server PORT=9000
```

Run REPL

```
make run-repl HOST=127.0.0.1 PORT=9000
```

Run benchmark (example)

```
make run-benchmark ROWS=10000000 CLIENTS=4
```

Run benchmark (unit-test mode)

```
make run-benchmark-unit
# or directly
.build/benchmark_flexql --unit-test
```

Optional reset

```
make reset-data
```

D Appendix D: Additional Benchmark Logs

Suggested log template

```
Date:
Machine:
Compiler + flags:
Server port:
Rows:
Clients:

Insert:
  elapsed_ms:
  throughput_rows_per_sec:

Full scan:
  elapsed_ms:
  total_rows_returned:

Primary-key lookup:
  elapsed_ms:
  total_rows_returned:

Unit tests:
  passed:
```

```
  failed:
```

Representative sample

```
Rows: 10000000
Clients: 4

Insert:
  elapsed = 28318 ms
  throughput = 353132 rows/sec

Full scan:
  elapsed = 18660 ms
  total_rows = 40000000

Primary-key lookup:
  elapsed = 5 ms
  total_rows = 4
```